

Experiment 3: Bubble Sort with Big-O Analysis

Aim

To write a Python program to **sort an array using Bubble Sort** and analyze its Big-O time complexity.

Algorithm

1. Start.
2. Read an array of elements.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the comparison for all adjacent elements.
6. After each pass, the largest unsorted element moves to its correct position.
7. Repeat until the array is sorted.
8. Display the sorted array.
9. Stop.

Source Code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

arr = [64, 34, 25, 12, 22, 11, 90]

print("Before Sorting:", arr)

bubble_sort(arr)

print("After Sorting:", arr)
```

Output :

Output

```
Before Sorting: [64, 34, 25, 12, 22, 11, 90]  
After Sorting: [11, 12, 22, 25, 34, 64, 90]
```

Result :

Thus, the Python program was successfully executed and the given array was successfully sorted in **ascending order** using Bubble Sort.